



“IS221” Bliss Flower Boutique System

Serial#	Section #	NAME	ID	Email
1	4w4	Fajr Omar Alrubeaan	445008461	445008461@pnu.edu.sa
2	4w4	Sadeem Tariq Alsubhi	445008366	445008366@pnu.edu.sa
3	4w4	AlDanah Naif AlQahtani	445008522	445008522@pnu.edu.sa
4	4w4	Shujon Ali Alshehri	445008413	445008413@pnu.edu.sa

Group: 3
Supervised By: D.Sammar Aldbass

➤ Project Introduction:

BLISS FLOWER BOUTIQUE: is a floral business with locations across the region, renowned for its exceptional customer service. The boutique specializes in creating custom floral arrangements for various occasions, including weddings, anniversaries, and corporate events. In addition to bespoke bouquets, BLISS offers delivery services, subscription options, and consultations for event planning. Customers can place orders through knowledgeable staff, ensuring personalized service. BLISS Flower Boutique caters to all clients but is especially suited for those seeking unique, short-term floral solutions for special events.

➤ Brief Description of the System

The **Bliss Flower Boutique System (BFBS)** is a database-driven solution designed to streamline customer orders, inventory management, and staff operations. It consists of the following core components:

Key Features:

1. User Roles & Permissions:

- **Admin:** Full access to all tables and operations.
- **Manager:** Can manage staff, Product, and customer orders.
- **Cashier:** Can process customer orders and generate bills.

2. Database Tables:

- **Staff:** Stores employee information (ID, name, contact details, salary).
- **Customer:** Maintains customer records (ID, name, contact details, address).
- **Order_:** Tracks customer orders (ID, customer, order date, status).
- **Product:** Manages product stock (ID, name, category, price, quantity).
- **Order_Product:** Links orders to products, tracking purchased items.
- **Bill:** Stores billing details for completed transactions.

3. Stored Procedures, Functions, and Triggers:

- **Stored Procedure:** **UpdateProductStock**(stored procedure updates the quantity of a product in the Order_Product table by a specified amount).
- **Function:** **GetStaffSalaryByID**(function returns the salary of a staff member based on their staff ID).

- **Trigger: SetDefaultOrderStatus** (automatically sets the order status to 'Pending' if the status is not provided when a new order is inserted into the Order_ table).

4. Query Optimization Techniques:

- **Indexing: Phone_Num** (Indexing speeds up query performance by allowing faster searches on indexed columns, reducing the need to scan entire tables).
- **Join Optimization:** Uses JOIN operations instead of subqueries for faster data retrieval between tables (**Customer, Order_, Product**).

System Objectives:

- **Ensure data accuracy** by preventing inventory(Product Table) errors and duplicate orders.
- **Enhance efficiency** in managing customer orders, staff, and Product.
- **Secure system data** with role-based access controls.
- **Optimize database performance** for faster query execution and retrieval.

All Tables

```
Run SQL Command Line
SQL> SELECT * FROM Staff;

STAFF PHONE_NUM SALARY EMAIL
-----
S01 050134567 3300 fajrAlrubeaan@example.com
S02 0502345678 3200 sadeemAlsubhi@example.com
S03 0503456789 3100 shujonAlshehri@example.com

SQL> SELECT * FROM Customer;

CUSTO EMAIL PHONE_NUM LAST_N FIRST_N CITY ZIP_CODE STREET STAFF
-----
C111 GhadiAldosari@example.com 0504567890 Aldosari Ghadi Riyadh 12345 123 Kawadir St S01
C121 GhedafArefein@example.com 0505678900 Arefein Ghedaf Abha 23456 456 Sahaba St S02
C131 SalehAlotaibi@example.com 0506789012 Alotaibi Saleh Dammam 34567 789 shabarqa St S03

SQL> SELECT * FROM Order_;

ORD CUSTO ORDER_DA STATUS TOTAL_PRICE
-----
O11 C111 01/11/24 Pending 150
O12 C121 15/11/24 Completed 200
O13 C131 07/09/24 Shipped 175
O15 11/04/25 Pending

SQL> SELECT * FROM Product;

PR PRODUCT_TYPE PRICE
-----
p1 Rose Bouquet 50
p2 Tulip Arrangement 75
p3 Lily Centerpiece 60

SQL> SELECT * FROM Order_Product;

ORD PR QUANTITY
-----
O11 p1 2
O12 p2 5
O13 p3 15

SQL> SELECT * FROM Bill;

ORD RECEIPT_NUMBER TOTAL_COST PAYMENT_TYPE CARD_INFO STAFF
-----
O11 R001 150 Credit Card **** * 1234 S01
O12 R002 200 Debit Card **** * 5678 S02
O13 R003 175 Cash S03

SQL>
```

1- Create Admin user with Role:

```
SQL> DROP ROLE ADMIN_ROLE;
Role dropped.
SQL> CREATE ROLE ADMIN_ROLE;
Role created.
SQL> GRANT CONNECT, RESOURCE TO ADMIN_ROLE;
Grant succeeded.
SQL> GRANT DBA TO ADMIN_ROLE;
Grant succeeded.
SQL>
SQL> DROP USER admin_user CASCADE;
User dropped.
SQL> CREATE USER admin_user IDENTIFIED BY admin123;
User created.
SQL> GRANT ADMIN_ROLE TO admin_user;
Grant succeeded.
```

2- Create Manager user with Role:

```
SQL>
SQL> DROP ROLE MANAGER_ROLE;
Role dropped.
SQL> CREATE ROLE MANAGER_ROLE;
Role created.
SQL> GRANT CONNECT, RESOURCE TO MANAGER_ROLE;
Grant succeeded.
SQL> GRANT SELECT, INSERT, UPDATE ON Staff TO MANAGER_ROLE;
Grant succeeded.
SQL> GRANT SELECT, INSERT, UPDATE ON Order_ TO MANAGER_ROLE;
Grant succeeded.
SQL> GRANT SELECT, INSERT, UPDATE ON Product TO MANAGER_ROLE;
Grant succeeded.
SQL>
SQL> DROP USER manager_user CASCADE;
User dropped.
SQL> CREATE USER manager_user IDENTIFIED BY manager123;
User created.
SQL> GRANT MANAGER_ROLE TO manager_user;
Grant succeeded.
```



3- Create **Cashier user** with Role:

```
SQL> DROP ROLE Cashier_Role;
Role dropped.
SQL> CREATE ROLE Cashier_Role;
Role created.
SQL> GRANT CONNECT TO Cashier_Role;
Grant succeeded.
SQL> GRANT SELECT, INSERT ON Order_ TO Cashier_Role;
Grant succeeded.
SQL> GRANT SELECT ON Customer TO Cashier_Role;
Grant succeeded.
SQL>
SQL> DROP USER cashier_user CASCADE;
User dropped.
SQL> CREATE USER cashier_user IDENTIFIED BY cashier123;
User created.
SQL> GRANT Cashier_Role TO cashier_user;
Grant succeeded.
```

Display (All) Users and Roles Assigned to Them

```
SQL> SELECT GRANTEE AS user_name,
2         GRANTED_ROLE AS role_name
3       FROM DBA_ROLE_PRIVS
4      ORDER BY user_name, role_name;
```

USER_NAME	ROLE_NAME
ADMIN_ROLE	CONNECT
ADMIN_ROLE	RESOURCE
ADMIN_USER	ADMIN_ROLE
APEX_040000	CONNECT
APEX_040000	RESOURCE
CASHIER_ROLE	CONNECT
CASHIER_USER	CASHIER_ROLE
CTXSYS	CTXAPP
CTXSYS	RESOURCE
DATAPUMP_EXP_FULL_DATABASE	EXP_FULL_DATABASE
DATAPUMP_IMP_FULL_DATABASE	EXP_FULL_DATABASE

USER_NAME	ROLE_NAME
DATAPUMP_IMP_FULL_DATABASE	IMP_FULL_DATABASE
DBA	DATAPUMP_EXP_FULL_DATABASE
DBA	DATAPUMP_IMP_FULL_DATABASE
DBA	DELETE_CATALOG_ROLE
DBA	EXECUTE_CATALOG_ROLE
DBA	EXP_FULL_DATABASE
DBA	GATHER_SYSTEM_STATISTICS
DBA	IMP_FULL_DATABASE
DBA	PLUSTRACE
DBA	SCHEDULER_ADMIN
DBA	SELECT_CATALOG_ROLE

USER_NAME	ROLE_NAME
DBA	XDBADMIN
DBA	XDB_SET_INVOKER
DBSNMP	OEM_MONITOR
EXECUTE_CATALOG_ROLE	HS_ADMIN_EXECUTE_ROLE
EXP_FULL_DATABASE	EXECUTE_CATALOG_ROLE
EXP_FULL_DATABASE	SELECT_CATALOG_ROLE
HR	CONNECT
HR	RESOURCE
HS_ADMIN_ROLE	HS_ADMIN_EXECUTE_ROLE
HS_ADMIN_ROLE	HS_ADMIN_SELECT_ROLE
IMP_FULL_DATABASE	EXECUTE_CATALOG_ROLE

USER_NAME	ROLE_NAME
IMP_FULL_DATABASE	SELECT_CATALOG_ROLE
LOGSDBY_ADMINISTRATOR	RESOURCE
MANAGER_ROLE	CONNECT
MANAGER_USER	MANAGER_ROLE
MDSYS	CONNECT
MDSYS	RESOURCE
OEM_MONITOR	SELECT_CATALOG_ROLE
OUTLN	RESOURCE
SELECT_CATALOG_ROLE	HS_ADMIN_SELECT_ROLE
SYS	ADM_PARALLEL_EXECUTE_TASK
SYS	APEX_ADMINISTRATOR_ROLE

USER_NAME	ROLE_NAME
SYS	AQ_ADMINISTRATOR_ROLE
SYS	AQ_USER_ROLE
SYS	AUTHENTICATEDUSER
SYS	CONNECT
SYS	CTXAPP
SYS	DATAPUMP_EXP_FULL_DATABASE
SYS	DATAPUMP_IMP_FULL_DATABASE
SYS	DBA
SYS	DBFS_ROLE
SYS	DELETE_CATALOG_ROLE
SYS	EXECUTE_CATALOG_ROLE

USER_NAME	ROLE_NAME
SYS	EXP_FULL_DATABASE
SYS	GATHER_SYSTEM_STATISTICS
SYS	HS_ADMIN_EXECUTE_ROLE
SYS	HS_ADMIN_ROLE
SYS	HS_ADMIN_SELECT_ROLE
SYS	IMP_FULL_DATABASE
SYS	LOGSDBY_ADMINISTRATOR
SYS	OEM_ADVISOR
SYS	OEM_MONITOR
SYS	PLUSTRACE
SYS	RECOVERY_CATALOG_OWNER

USER_NAME	ROLE_NAME
SYS	RESOURCE
SYS	SCHEDULER_ADMIN
SYS	SELECT_CATALOG_ROLE
SYS	XDBADMIN
SYS	XDB_SET_INVOKER
SYS	XDB_WEBSERVICES
SYS	XDB_WEBSERVICES_OVER_HTTP
SYS	XDB_WEBSERVICES_WITH_PUBLIC
SYSTEM	ADMIN_ROLE
SYSTEM	AQ_ADMINISTRATOR_ROLE
SYSTEM	CASHIER_ROLE

USER_NAME	ROLE_NAME
SYSTEM	DBA
SYSTEM	MANAGER_ROLE
XDB	CTXAPP
XDB	RESOURCE

81 rows selected.

Show (privileges) for each role has **on each tables** in the database.

```
SQL> --query helps identify (privileges) for each role has on which tables in the database.--
SQL> SELECT grantee, table_name, privilege
2 FROM dba_tab_privs
3 WHERE grantee IN ('ADMIN_ROLE', 'MANAGER_ROLE', 'CASHIER_ROLE')
4 ORDER BY grantee, table_name;
```

GRANTEE	TABLE_NAME	PRIVILEGE
CASHIER_ROLE	CUSTOMER	SELECT
CASHIER_ROLE	ORDER_	INSERT
CASHIER_ROLE	ORDER_	SELECT
MANAGER_ROLE	ORDER_	INSERT
MANAGER_ROLE	ORDER_	UPDATE
MANAGER_ROLE	ORDER_	SELECT
MANAGER_ROLE	PRODUCT	INSERT
MANAGER_ROLE	PRODUCT	SELECT
MANAGER_ROLE	PRODUCT	UPDATE
MANAGER_ROLE	STAFF	SELECT
MANAGER_ROLE	STAFF	INSERT
MANAGER_ROLE	STAFF	UPDATE

12 rows selected.

SQL> |

Show which users have been **assigned which roles** from the database's role management system

```
SQL> SELECT
2 GRANTEE AS USERNAME,
3 GRANTED_ROLE
4 FROM
5 DBA_ROLE_PRIVS
6 WHERE
7 GRANTED_ROLE IN ('ADMIN_ROLE', 'MANAGER_ROLE', 'CASHIER_ROLE')
8 AND GRANTEE IN ('ADMIN_USER', 'MANAGER_USER', 'CASHIER_USER');
```

USERNAME	GRANTED_ROLE
ADMIN_USER	ADMIN_ROLE
CASHIER_USER	CASHIER_ROLE
MANAGER_USER	MANAGER_ROLE

SQL> |



Index Creation

```
SQL> DROP INDEX idx_customer_phone;
Index dropped.
SQL> CREATE INDEX idx_customer_phone ON Customer(Phone_Num);
Index created.
SQL>
SQL> SELECT * FROM Customer WHERE Phone_Num = '0506789012';
```

CUST_EMAIL	PHONE_NUM	LAST_N	FIRST_N	CITY	ZIP_CODE	STREET	STA
C131 SalehAlotaibi@example.com	0506789012	Alotaibi	Saleh	Dammam	34567	789 shabarqa St	S03

```
SQL> |
```

Join Optimization

```
SQL> SELECT c.First_N, c.Last_N, o.Order_ID, p.Product_Type
2 FROM Customer c, Order_ o, Order_Product op, Product p
3 WHERE c.Customer_ID = o.Customer_ID
4 AND o.Order_ID = op.Order_ID
5 AND op.Product_ID = p.Product_ID
6 AND p.Product_Type = 'Tulip Arrangement';
```

FIRST_N	LAST_N	ORD	PRODUCT_TYPE
Ghedaf	Arefein	012	Tulip Arrangement

Stored Procedure: UpdateProductStock

Used by managers/admins to update product quantity (e.g., after a new stock delivery).

```
SQL>
SQL> CREATE OR REPLACE PROCEDURE UpdateProductStock (
  2     p_product_id IN NUMBER,
  3     p_added_quantity IN NUMBER
  4 )
  5 IS
  6 BEGIN
  7     UPDATE Order_Product
  8     SET Quantity = Quantity + p_added_quantity
  9     WHERE Product_ID = p_product_id;
 10
 11 END;
 12 /
```

Procedure created.

```
SQL>
SQL> BEGIN
  2     UpdateProductStock('p3', 5);
  3 END;
  4 /
```

PL/SQL procedure successfully completed.

```
SQL> SELECT * FROM Order_Product WHERE Product_ID = 'p3';
```

ORD	PR	QUANTITY
---	--	-----
013	p3	15

```
SQL>
```

Function: GetStaffSalaryByIDv

--Returns the salary of a staff member based on their ID.

```
SQL> CREATE OR REPLACE FUNCTION GetStaffSalaryByIDv(
  2     p_staff_id IN CHAR
  3 )
  4 RETURN NUMBER AS
  5     staff_salary NUMBER(10,2);
  6 BEGIN
  7     SELECT Salary INTO staff_salary
  8     FROM Staff
  9     WHERE Staff_ID = p_staff_id;
 10
 11     RETURN staff_salary;
 12 END;
 13 /
```

Function created.

```
SQL>
SQL> SET SERVEROUTPUT ON;
SQL> DECLARE
  2     staff_salary NUMBER(10,2);
  3     p_staff_id CHAR(10);
  4 BEGIN
  5     p_staff_id :=& p_staff_id;
  6     staff_salary := GetStaffSalaryByIDv(p_staff_id);
  7     DBMS_OUTPUT.PUT_LINE('The salary based on ID ' || p_staff_id || ' is: ' || staff_salary);
  8 END;
  9 /
```

```
Enter value for p_staff_id: 'S02'
old  5: p_staff_id :=& p_staff_id;
new  5: p_staff_id :='S02';
The salary based on ID S02          is: 3200
```

PL/SQL procedure successfully completed.

```
SQL> SELECT * FROM Staff;
```

STA	PHONE_NUM	SALARY	EMAIL
S01	050134567	3300	fajrAlrubeaan@example.com
S02	0502345678	3200	sadeemAlsubhi@example.com
S03	0503456789	3100	shujonAlshehri@example.com

Trigger: SetDefaultOrderStatus

--Automatically sets the order status to 'Pending' if it's not provided
when a new order is inserted.

```
SQL> CREATE OR REPLACE TRIGGER SetDefaultOrderStatus
2  BEFORE INSERT ON Order_
3  FOR EACH ROW
4  BEGIN
5      IF :NEW.status IS NULL THEN
6          :NEW.status := 'Pending';
7      END IF;
8  END;
9  /
```

Trigger created.

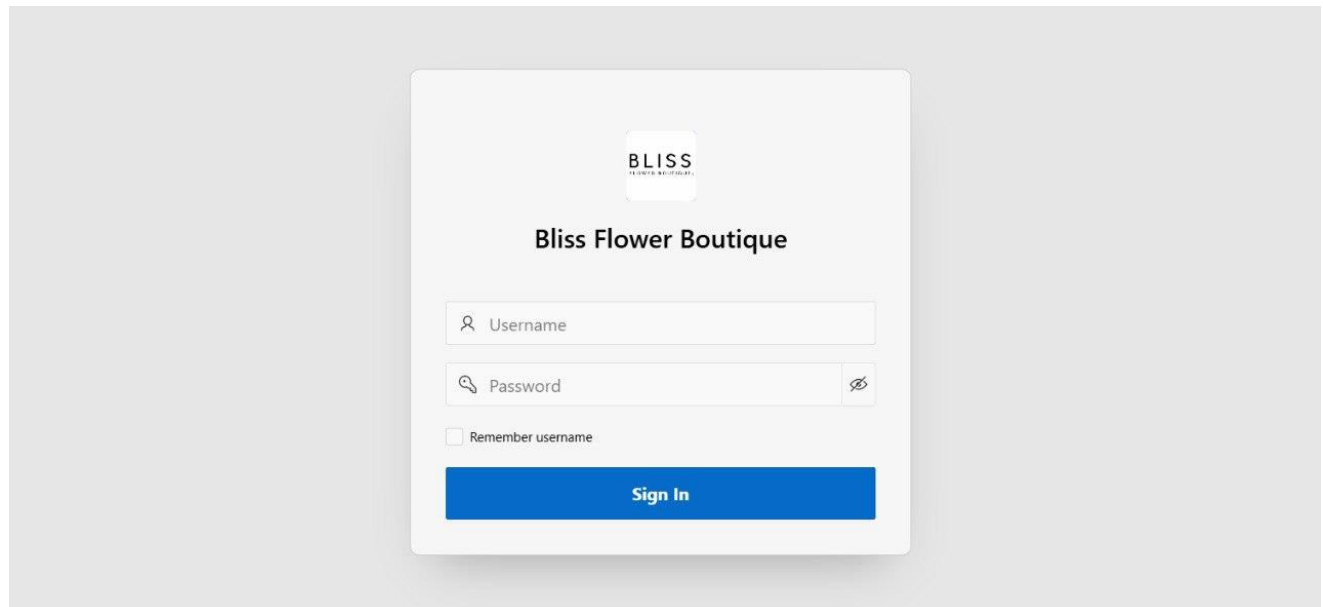
```
SQL>
SQL> INSERT INTO Order_ (Order_ID, Order_Date)
2  VALUES ('014', TO_DATE('2025-04-11', 'YYYY-MM-DD'));
```

1 row created.

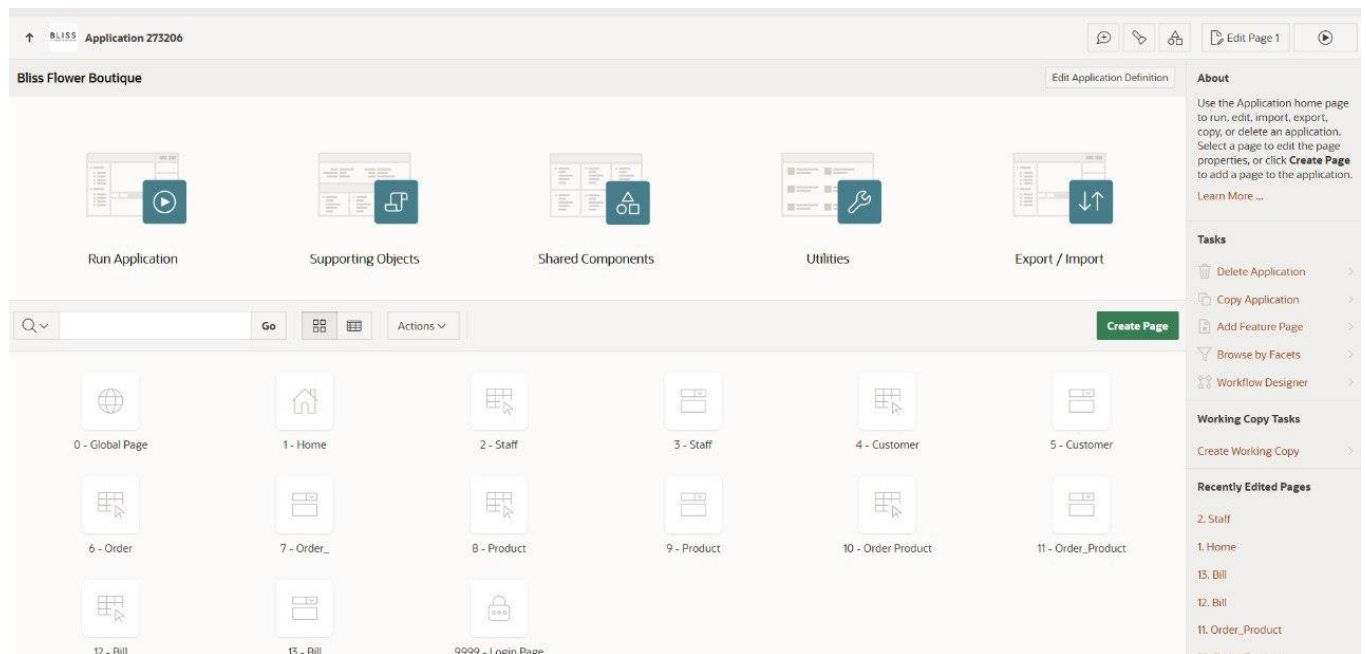
```
SQL> SELECT * FROM Order_ WHERE Order_ID = '014';
```

ORD	CUST	ORDER_DAT	STATUS	TOTAL_PRICE
014		11-APR-25	Pending	

--Forms and Report interface --



The image shows a login form for 'Bliss Flower Boutique'. The form is centered on a light gray background. It features a logo at the top, followed by the title 'Bliss Flower Boutique'. Below the title are two input fields: 'Username' and 'Password'. The 'Password' field has a toggle icon for visibility. There is a checkbox labeled 'Remember username' and a blue 'Sign In' button at the bottom.



The image displays the main interface of the 'Bliss Flower Boutique' application. The top bar shows the application name and a search bar. Below the top bar, there are five main action buttons: 'Run Application', 'Supporting Objects', 'Shared Components', 'Utilities', and 'Export / Import'. A search bar is located below these buttons. The main area contains a grid of application pages, each with a thumbnail and a label: '0 - Global Page', '1 - Home', '2 - Staff', '3 - Staff', '4 - Customer', '5 - Customer', '6 - Order', '7 - Order_', '8 - Product', '9 - Product', '10 - Order Product', '11 - Order_Product', '12 - Bill', '13 - Bill', and '9999 - Login Page'. On the right side, there is a sidebar with sections for 'About', 'Tasks', 'Working Copy Tasks', and 'Recently Edited Pages'.



Home Bliss Flower Boutique

Staff Customer Order_ Product Order_Product Bill

Staff Customer Order_ Product Order_Product Bill

Bliss Flower Boutique

Staff

Phone Num Salary Email

050134567	3,300	fajrAlnubeaan@example.com
0502345678	3,200	sadeemAlsubhi@example.com
0503456789	3,100	shujonAlshehri@example.com

1 - 3

Bliss Flower Boutique

Customer

Email Phone Num Last N First N City Zip Code Street Staff

GhadiAldosari@example.com	0504567890	Aldosari	Ghadi	Riyadh	12345	123 Kawadir St	050134567
GhedafArefein@example.com	0505678900	Arefein	Ghedaf	Abha	23456	456 Sahaba St	0502345678
SalehAlotaibi@example.com	0506789012	Alotaibi	Saleh	Dammam	34567	789 shabarka St	0503456789

1 - 3



Bliss Flower Boutique

Home

Staff

Customer

Order_

Product

Order_Product

Bill

Order

Q

Go

Actions

Create

	Customer	Order Date	Status	Total Price
	GhadiAldosari@example.com	4/11/2025	Pending	
	GhedatArefein@example.com	11/1/2024	Pending	150
	GhedatArefein@example.com	11/15/2024	Completed	200
	SalehAlotaibi@example.com	9/7/2024	Shipped	175
				1 - 4

Bliss Flower Boutique

Home

Staff

Customer

Order_

Product

Order_Product

Bill

Product

Q

Go

Actions

Create

	Product Type	Price
	Rose Bouquet	50
	Tulip Arrangement	75
	Lily Centerpiece	60
		1 - 3

Bliss Flower Boutique

Home

Staff

Customer

Order_

Product

Order_Product

Bill

Bill

Q

Go

Actions

Create

	Total Cost	Payment Type	Card Info	Staff
	150	Credit Card	**** * 1234	050134567
	200	Debit Card	**** * 5678	0502345678
	175	Cash		0503456789
				1 - 3



Bliss Flower Boutique 119m7g13@gmail.com

Home Staff Customer Order_ Product **Order_Product** Bill

Order Product

Q Go Actions Create

	Quantity
	2
	5
	15
	1 - 3

Logout
EMPLOYEES

EMPLOYEES > EMPLOYEES

Cancel Delete Apply Changes

EMPLOYEES

First Name Adam
Last Name Fripp
Email AFRIPP
Phone Number 650.123.2234
Hire Date 10/04/97
Job Id ST_MAN
Salary 8400
Commission Pct
Manager Id 100
Department Id 50

HR

Edit Application Edit Page 2 Create Session Debug Show Edit Links

EMPLOYEES

Reset Create

Search Display 100 Go

First Name	Last Name	Email	Phone Number	Department Id	Job Id	Salary	Commission Pct	Manager Id	Hire Date
Adam	Fripp	AFRIPP	650.123.2234	50	ST_MAN	8400		100	10/04/97
Alana	Walsh	AWALSH	650.507.9811	50	SH_CLERK	3100		124	24/04/98
Alberto	Errazuriz	AERRAZUR	011.44.1344.429278	80	SA_MAN	13000	.3	100	10/03/97
Alexander	Hunold	AHUNOLD	590.423.4567	60	IT_PROG	9000		102	03/01/90
Alexander	Khoo	AKHOO	515.127.4562	30	PU_CLERK	3100		114	18/05/95
Alexis	Bull	ABULL	650.509.2876	50	SH_CLERK	4100		121	20/02/97
Alian	McEwen	AMCEWEN	011.44.1345.829268	80	SA_REP	9000	.35	146	01/08/96
Alyssa	Hutton	AHUTTON	011.44.1644.429266	80	SA_REP	8800	.25	149	19/03/97
Amir	Banda	ABANDA	011.44.1346.729268	80	SA_REP	6200	.1	147	21/04/00
Anthony	Cabrio	ACABRIO	650.509.4876	50	SH_CLERK	3000		121	07/02/99
Britney	Everett	BEVERETT	650.501.2876	50	SH_CLERK	3900		123	03/03/97
Bruce	Ernst	BERNST	590.423.4568	60	IT_PROG	6000		103	21/05/91
Charles	Johnson	CJOHNSON	011.44.1644.429262	80	SA_REP	6200	.1	149	04/01/00
Christopher	Olsen	COlsen	011.44.1344.498718	80	SA_REP	8000	.2	145	30/03/98
Clara	Vishney	CVISHNEY	011.44.1346.129268	80	SA_REP	10500	.25	147	11/11/97
Curtis	Davies	CDAVIES	650.121.2994	50	ST_CLERK	3100		124	29/01/97
Daniel	Faviet	DFAVIET	515.124.4169	100	FI_ACCOUNT	9000		108	16/08/94
Danielle	Greene	DGREENE	011.44.1346.229268	80	SA_REP	9500	.15	147	19/03/99
David	Austin	DAUSTIN	590.423.4569	60	IT_PROG	4800		103	25/06/97
David	Bernstein	DBERNSTE	011.44.1344.345268	80	SA_REP	9500	.25	145	24/03/97
David	Lee	DLEE	011.44.1346.529268	80	SA_REP	6800	.1	147	23/02/00
Den	Danahelu	DDANAHELU	515.127.4561	50	SA_MAN	11000		100	07/11/94